

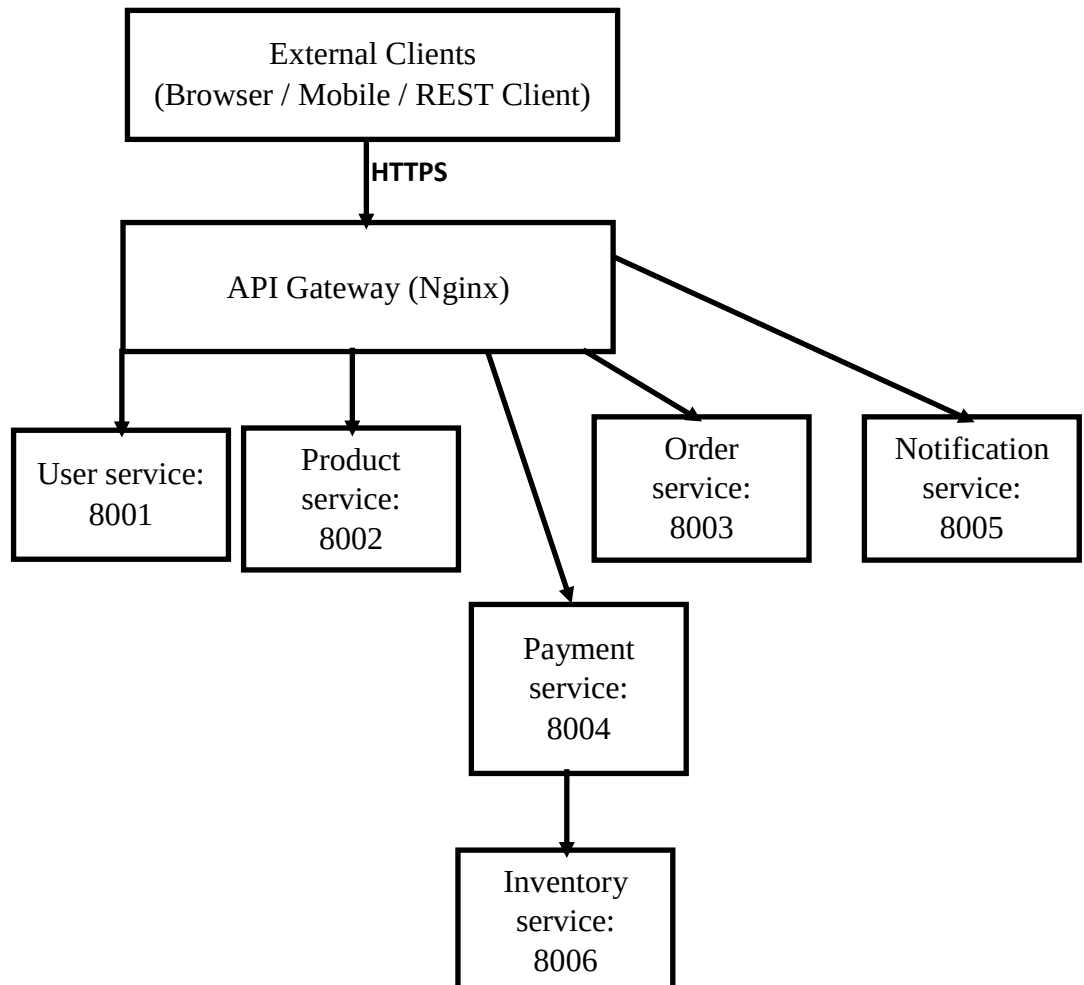
Workshop 3: DevSecOps

1. System Architecture Overview: A High-Level Description

SecureShop is a lightweight e-commerce platform based on a microservices architecture. The system consists of six microservices:

	Service	Key Responsibilities
1	User Service	Registration, login, JWT issuance, profile management
2	Product Service	Product catalogue, search, categories
3	Order Service	Cart management, order lifecycle
4	Payment Service	Payment initiation, transaction records
5	Notification Service	Email/SMS dispatch via message queue
6	Inventory Service	Stock levels, reservation, release

The following figure demonstrates a simple view of the system:



In addition to the six services, a simple API gateway provides a single entry point for all services. Its key responsibilities are:

- JWT authentication plugin (validates tokens issued by User Service).
- Rate limiting (for example, 100 req/min per IP).
- Request Routing
-

2. Inter-Service Communication

- **Synchronous:** REST over HTTP.
- **Asynchronous (optional):** RabbitMQ for Order → Notification events.

3. Infrastructure

All services are containerised with **Docker** and orchestrated locally with **Docker Compose**. The same `docker-compose.yml` is used as the deployment target for DAST testing in CI.

4. Technology Stack

4.1 Application Stack

Component	Technology
Container runtime	Docker
Orchestration (local)	Docker Compose
API Gateway	Nginx
Message Broker (optional)	RabbitMQ
CI/CD	GitHub Actions

4.2 DevSecOps Toolchain

To try different SAST tools, it is preferable to use at least two different programming languages when developing the microservices (e.g., Python/Node.js, Python/Java, ...). The following table contains examples of suggested tools categorised by DevSecOps practices:

Category	Tool	Rationale
SAST Python	Bandit	Purpose-built for Python security anti-patterns
SAST JS/TS	Semgrep OSS	Multi-language, extensible rules, SARIF output
SAST Java	SpotBugs + FindSecBugs plugin	Mature bytecode analysis for Java

	SonarQube	Open source and fully usable with GitHub Actions
SAST (multi languages)	Semgrep OSS	Covers all languages with community rule packs
SCA	OWASP Dependency-Check	CVE matching for all dependency types
	Trivy	Fast, accurate, broad ecosystem support
Secret scanning	Gitleaks	Git history + staged changes scanning
	TruffleHog	Entropy + pattern scanning in commits
Container scanning	Trivy	OS packages + app dependencies in images
	Grype	Anchore's standalone scanner, useful 2nd opinion
DAST	OWASP ZAP	Industry-standard; automation via <code>`zap\_baseline.py`</code>
IaC scanning	Checkov	Dockerfile, docker-compose, k8s, Terraform
Secrets runtime	HashiCorp Vault (dev mode)	Industry standard; dev mode for learning
SBOM generation	Syft	Generates CycloneDX/SPDX SBOM
Linting/quality	Hadolint	Dockerfile best-practice linter
Reporting	GitHub Security tab (SARIF)	Free; integrates with CodeQL upload action

5. The requested work:

- Step 0- Threat Modeling:** This step is strongly recommended.
- Step 1-Project Setup & Repository Structure:** Create a **single GitHub repository**.
- Step 2-SAST (Required):** analyses source code without executing it.
- Step 3-SCA (Required):** identifies known vulnerabilities (CVEs) in third-party dependencies.
- Step 4-Secrets Detection:** Hard-coded secrets (API keys, passwords, tokens) are among the most common and damaging vulnerabilities found in real codebases.
- Step 5-Container Image Scanning:** Container images bundle the application code with an OS base layer and system libraries. Both layers can contain CVEs. We must scan the **final built image**, not just the source code.
- Step 6-DAST (Required):** involves interacting with a running application to discover vulnerabilities such as SQL injection, XSS, and authentication weaknesses that SAST cannot reliably detect.

8. Step 7-Infrastructure-as-Code Security Scanning (optional): (Dockerfiles, docker-compose) can contain misconfigurations that create security risks.

9. Step 8-Secrets Management at Runtime (optional): secrets in code or configuration:

- Hard-coded DB password in `docker-compose.yml`
- JWT secret in the source code
- RabbitMQ credentials in environment variables
- SMTP password in Notification Service config

10. Step 9-Assembling the Full CI/CD Pipeline:

- Complete Pipeline Architecture
- Complete Orchestrator Workflow

11. Step 10-Security Dashboard & Reporting:

- GitHub Security Tab: All SARIF uploads become visible in Repository → Security → Code Scanning. This gives:
 - ✓ Aggregated findings across all tools
 - ✓ Ability to dismiss/suppress findings with justification
 - ✓ Trend tracking over time
- Write a Security Report: propose a template that summarises the realised work (Executive Summary, Methodology, Findings Summary with vulnerabilities prioritisation and ranking, Threat Model vs. Findings Correlation, ...).

6. Deliverables

	Deliverable	Format
D1	Threat Model (DFD L0+L1 + STRIDE worksheet)	PDF / Threat Dragon JSON
D2	Working microservices application (Docker Compose up)	GitHub repo
D3	CI/CD pipeline (all stages green or documented)	GitHub Actions + YAML
D4	Tool findings tables (SAST, SCA, Secrets, Container, DAST, IaC)	Markdown / Excel
D5	Security report (a proposed template)	PDF
D6 (optional)	Live demo (10-min)	Presentation